

Session Report

Spatial Graph, Collision & Interactive Visualizer

AIA26 Studio - Team 03

Industrial Spatial Flow Agent

Date: May 22, 2026

Files created: spatial_graph.py, test_spatial_graph.py, visualize_interactive.py

Files modified: collision.py, graph.py, reason.py, prompts.py, add_objects.py

New directory: python/view_graph/ (HTML output + local vis.js)

Docs: CLAUDE.md, session report PDF

1. Session Summary

The spatial graph layer was implemented and integrated into the LangGraph pipeline. Key changes:

1. Created `spatial_graph.py` - a pure NetworkX module that builds a MultiGraph from layout JSON (rooms, doors, walls, windows, furniture, mep) with typed nodes and edges (`contained_in`, `door_connects`, `adjacent`, `near`, `near_wall`, `near_window`).
2. Integrated the graph into the agent pipeline: AgentState fields, `enrich_graph_node` (runs after all 5 analysis tools), correction message injection for LLM auto-fix, graph rebuild after each placement.
3. Fixed collision clearance (`collision.py`): Voronoi boundary method computes real surface-to-surface gaps instead of constant 0.1m.
4. Added walls and windows as graph nodes with `near_wall/near_window` proximity edges. Walls filtered from FINDINGS (structural, not movable).
5. Fixed `clearance_ok` to use `deficit_m <= 0` instead of checking dict presence.
6. Created `test_spatial_graph.py` standalone visualizer with edge descriptions in legend.
7. Injected `spatial_graph_text` into LLM context (`reason.py`) and added SPATIAL GRAPH section to SYSTEM_PROMPT (`prompts.py`).

2. Collision Clearance Fix (Voronoi Boundary Method)

The Problem

All 19 furniture/MEP objects in the test layout showed identical clearance values: 'has 0.1m, needs 0.4m' with a deficit of -0.3m. This happened because `min_clearance_m` was computed inside the violation cell loop, tracking the minimum `dist * cell_size` for cells attributed to each object. Since `dist` measures BFS distance from any free cell to the nearest obstacle surface, and the cells immediately adjacent to any object always have `dist=1` (= 0.1m at 0.10m grid resolution), every object bottomed out at 0.1m.

```
BEFORE (all identical):
CLEARANCE Toilet: has 0.1m, needs 0.4m -> move [...] 0.4m
CLEARANCE Sink: has 0.1m, needs 0.4m -> move [...] 0.4m
CLEARANCE Assembly Station 1: has 0.1m, needs 0.4m -> move [...] 0.4m
CLEARANCE cnc_machine: has 0.1m, needs 0.4m -> move [...] 0.4m
... (all 19 objects show 0.1m)
```

The Solution

After the violation cell loop, a Voronoi boundary scan computes the real gap between each object and its nearest other obstacle (wall, furniture, or MEP). The algorithm:

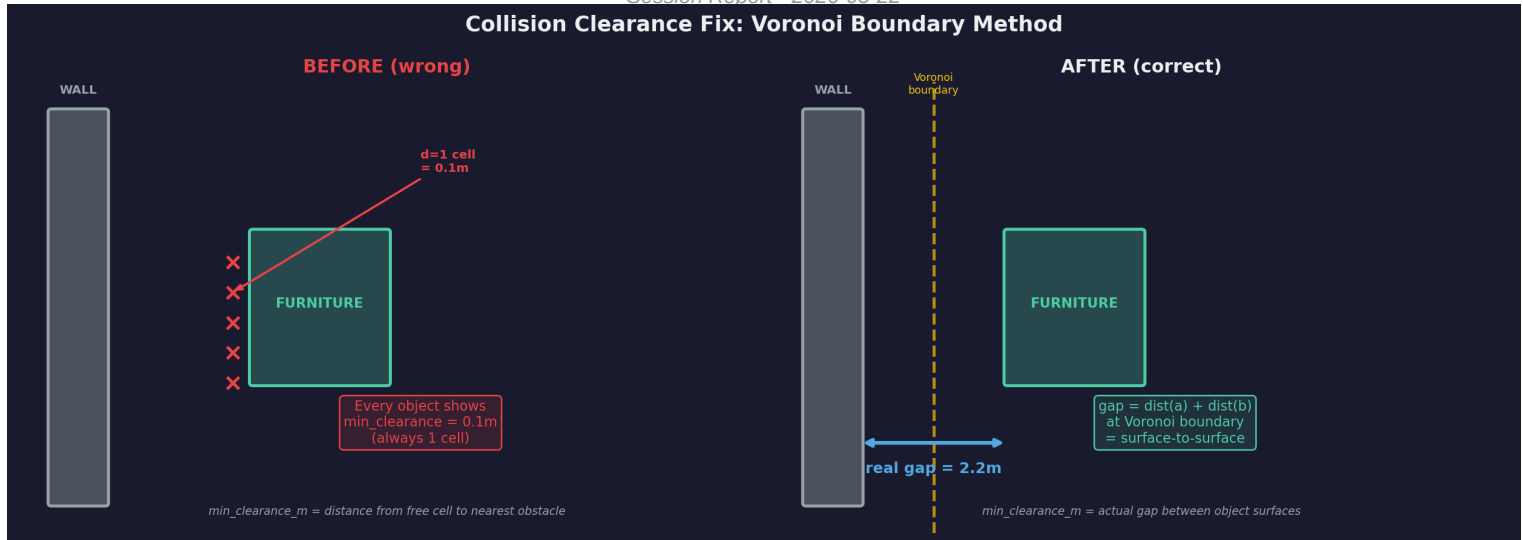
1. Iterate all free cells in the grid (`dist > 0`)
2. For each free cell, check its 4-connected neighbors
3. If a neighbor's nearest-obstacle attribution differs (`nearest[cell] != nearest[neighbor]`), we are at a Voronoi boundary between two obstacles
4. The gap at that boundary = $(\text{dist}[\text{cell}] + \text{dist}[\text{neighbor}]) * \text{cell_size}$
5. Track the minimum gap for each object across all its Voronoi boundaries

This gives the actual surface-to-surface distance between the object and its nearest neighbor, not the per-cell distance to any obstacle.

```
# Core of the Voronoi boundary scan:
for idx in range(total):
    d = dist[idx]
    if d <= 0: continue
    oi = nearest[idx]
    for neighbor in 4_connected(idx):
        dn = dist[neighbor]
        oj = nearest[neighbor]
        if oj != oi: # Voronoi boundary!
            gap = (d + dn) * cell_size
            obj_real_clearance[oi] = min(gap, ...)
            obj_real_clearance[oj] = min(gap, ...)
```

```
AFTER (differentiated per object):
CLEARANCE Toilet: has 0.35m, needs 0.4m -> move [...] 0.15m
Assembly Station 1: clearance=OK (gap 1.2m > 0.4m required)
cnc_machine: has 0.28m, needs 0.4m -> move [...] 0.22m
```

Collision Clearance Fix: Voronoi Boundary Method



No changes were needed in the Grasshopper component. The GH script independently re-runs the grid analysis for visualization only. The Python side (collision.py) is the authoritative source for all clearance values consumed by the agent and spatial graph.

3. Walls and Windows in the Spatial Graph

What was missing

The spatial graph previously included 4 element types: rooms, doors, furniture, and MEP. The layout JSON has 7 layers. Of the missing 3 (outline, structure, windows), structure and windows carry important spatial information:

- Structure (walls): 6 walls in the test layout (4 exterior load-bearing, 2 interior partitions). Essential for knowing if furniture is against a wall, which walls are structural vs. partition, and wall-proximity relationships.

What was added

New node types:

```
wall node:
  ntype='wall', name, wall_type (load-bearing/partition),
  material (concrete/drywall), length, p1, p2 (segment endpoints)

window node:
  ntype='window', name, window_type (awning/sliding/fixed),
  roomId, width, p1, p2 (segment endpoints)
+ contained_in edge to parent room
```

New edge types:

```
near_wall:
  furniture -> wall, distance_m (point-to-segment), threshold 3m
  Uses _point_to_segment_distance() for accurate projection

near_window:
  furniture -> window, same room only, same distance method
```

New helper function:

```
def _point_to_segment_distance(px, py, x1, y1, x2, y2):
    # Orthogonal projection of point onto segment,
    # clamped to endpoints. Returns shortest distance.
    t = clamp(0, 1, dot(P-A, B-A) / |B-A|^2)
    return dist(P, A + t*(B-A))
```

Serialization changes:

- STRUCTURE: section lists each wall with type and length

5. Updated LLM Serialization

The `serialize_for_llm()` output now includes **STRUCTURE** and **WINDOWS** sections. Below is the complete output from `industrial_005` (base layout, no analysis enrichment):

```
SPATIAL GRAPH (45 nodes, 43 edges)

ROOMS:
  room-1 "Bathroom" area=45.5m2
  room-2 "Clean Room" area=812.5m2
CONNECTIVITY:
  Bathroom <--door-1(0.9m)--> Clean Room
STRUCTURE:
  wall-1 "South Exterior Wall" load-bearing 52.0m
  wall-2 "East Exterior Wall" load-bearing 16.5m
  wall-3 "North Exterior Wall" load-bearing 52.0m
  wall-4 "West Exterior Wall" load-bearing 16.5m
  wall-5 "Interior Wall 5" partition 6.5m
  wall-6 "Interior Wall 6" partition 7.0m
WINDOWS: Clean Room: 16 (5x awning, 8x sliding, 3x fixed)
FURNITURE in Bathroom:
  furn-1 "Toilet" at(45.5,10.6)
  furn-2 "Sink" at(46.3,10.5)
FURNITURE in Clean Room:
  furn-3 "Assembly Station 1" at(7.1,10.5)
  furn-4 "Packaging Station 2" at(31.9,10.6)
  furn-5 "Packaging Station 3" at(14.3,5.5)
  furn-6 "Packaging Station 4" at(6.7,7.4)
  furn-7 "Conveyor Section 5" at(29.7,4.5)
  furn-8 "Labeling Station 6" at(25.8,3.1)
  furn-9 "Parts Bin Rack 7" at(9.0,5.3)
  furn-10 "Conveyor Section 8" at(18.8,12.8)
  furn-11 "Parts Bin Rack 9" at(19.0,2.5)
  furn-12 "Conveyor Section 10" at(39.0,12.3)
  furn-13 "Parts Bin Rack 11" at(28.5,9.5)
  furn-14 "QC Table 12" at(37.1,10.3)
MEP:
  mep-1 "HVAC Unit 1" system=hvac
  mep-2 "HVAC Unit 2" system=hvac
  mep-3 "Main Electrical Panel" system=electrical
  mep-4 "Plumbing Riser" system=plumbing
RELATIONS:
  Toilet --near(0.88m)--> Sink
  Toilet --near_wall(0.46m)--> Interior Wall 5
  Toilet --near_wall(0.56m)--> Interior Wall 6
  Sink --near_wall(1.34m)--> Interior Wall 5
  Sink --near_wall(0.48m)--> Interior Wall 6
  Parts Bin Rack 9 --near_wall(2.5m)--> South Exterior Wall
  Parts Bin Rack 9 --near_window(2.7m)--> Warehouse Window 4
  Conveyor Section 10 --near(2.82m)--> QC Table 12
```

6. Pipeline Integration (graph.py, reason.py, prompts.py, add_objects.py)

graph.py - AgentState + enrich_graph_node

Two new fields added to AgentState: `spatial_graph` (dict, node-link format) and `spatial_graph_text` (str, compact LLM serialization). Both use `_keep_last` reducer for parallel branch safety.

`enrich_graph_node` runs after reachability and before the `group2` routing decision. It:

1. Deserializes the graph from state
2. Calls `enrich_graph_from_analysis()` with all 5 analysis results
3. Prints ANSI-colored FINDINGS (red=clearance, yellow=unreachable/facing)
4. Walls filtered from FINDINGS (`_skip_ntypes = {'wall'}`)
5. If findings + objects placed: builds correction message with exact move vectors
6. Returns updated `spatial_graph` + `spatial_graph_text`

Wiring: `reachability` -> `enrich_graph` -> `_route_after_group2` -> {reason, scoring}

reason.py - Context injection

`spatial_graph_text` is injected into the LLM context before each call, alongside `space_config` and `profile_config`. The LLM sees the full graph topology and any issues with exact move vectors.

prompts.py - SPATIAL GRAPH section

Added a SPATIAL GRAPH section to `SYSTEM_PROMPT` instructing the LLM to check the ISSUES section for violations with exact move vectors, and to use `move_object` with those vectors instead of guessing new positions.

add_objects.py - Graph rebuild after placement

After both the MCP path and the fallback manual path, the spatial graph is rebuilt from scratch (base edges only, no analysis enrichment). Previous analysis edges no longer apply when positions change. Wrapped in `try/except` so graph failure doesn't break the placement pipeline.

7. clearance_ok Fix + Wall Filtering

Two related fixes ensure walls don't pollute the FINDINGS output:

1. clearance_ok evaluation: Previously set to False whenever a clearance_violation dict existed, regardless of deficit value. Fixed to check deficit_m <= 0.

```
# BEFORE:
if cv:
    G.nodes[oid]['clearance_ok'] = False # always False if cv exists

# AFTER:
if cv:
    deficit = cv.get('deficit_m', 0)
    G.nodes[oid]['clearance_ok'] = deficit <= 0 # OK if no real deficit
```

2. Wall filtering: After adding wall nodes to the graph, collision enrichment started setting clearance attributes on walls (because wall IDs now exist in G). Walls show 0.2m clearance (= wall thickness at Voronoi boundary), which is not actionable. Fixed with _skip_ntypes = {'wall'} in three locations:

- enrich_graph_from_analysis() in spatial_graph.py - skips collision enrichment for walls

The test_spatial_graph.py legend now shows a short description for each edge type, making the visualization self-documenting:

```
EDGE_DESCRIPTIONS = {
    'contained_in': 'element belongs to room',
    'door_connects': 'door links to room',
    'adjacent': 'rooms share a door',
    'near': 'furniture < 3m apart',
    'near_wall': 'furniture < 3m from wall',
    'near_window': 'furniture < 3m from window',
    'blocks': 'object blocks access to another',
    'sightline': 'direct line of sight between objects',
    'path': 'navigable route with distance',
}
```

Legend format: 'edge_type (count) - description'. Example: 'near_wall (12) - furniture < 3m from wall'.

9. Interactive Graph Visualizer (visualize_interactive.py)

A complete rewrite of the graph visualizer using raw HTML + vis.js 9.1.2 (dropped pyvis). Apple-minimalist aesthetic with muted colors, glass-morphism panels, and fixed architectural node positions reflecting the actual floor plan layout.

Core Features

- Architectural positions: nodes placed at real layout coordinates (flipped Y), physics disabled, fixed:true - no force-directed bouncing

Live Refresh System

The visualizer auto-refreshes when the pipeline regenerates the HTML file. A background HTTP server daemon (port 7477) with CORS headers serves the file. Two detection modes:

1. HTTP smart detection: fetch() compares PAGE_TS millisecond timestamp embedded in HTML. Only reloads when the timestamp changes.
2. Blind reload fallback: for file:// origins where fetch() is blocked by same-origin policy. Uses location.reload() with adaptive backoff (2-10s) via sessionStorage. Preserves viewport (zoom/pan) across reloads via network.moveTo().

Pipeline Integration

The graph auto-updates at three pipeline points:

1. Startup (_build_initial_state in graph.py): generates initial HTML from base layout
2. After placement (add_objects.py): rebuilds graph, highlights new/moved furniture via viz_highlight_ids state field
3. After enrichment (enrich_graph_node in graph.py): marks enrichment edges as new, merges carry-over highlights from viz_highlight_ids

viz_highlight_ids is an Annotated[list[str] | None, _keep_last] field in AgentState that carries furniture IDs highlighted by add_objects into enrich_graph_node.

File Structure

```
python/
  visualize_interactive.py      # Main visualizer module (~1190 lines)
  view_graph/
    spatial_graph_interactive.html # Generated HTML (auto-updated)
    lib/vis-9.1.2/               # Local vis.js 9.1.2
    lib/bindings/                 # vis.js bindings
    lib/tom-select/               # Tom Select library
```

10. All Files Modified

spatial_graph.py -- NEW - Spatial graph module

Pure NetworkX module (~570 lines). build_graph_from_layout(), enrich_graph_from_analysis(), serialize_for_llm(), graph_to_dict(), dict_to_graph(). Point-to-segment distance for wall/window proximity. Walls skipped in collision enrichment. clearance_ok based on deficit_m <= 0.

test_spatial_graph.py -- NEW - Standalone visualizer

Matplotlib dark theme, nodes colored by type, edges styled by type. Legend with edge descriptions. Modes: --session, layout name, --all.

graph.py -- Spatial graph integration

AgentState: +2 fields (spatial_graph, spatial_graph_text). enrich_graph_node inline. _build_correction_message(). Rewired reachability -> enrich_graph -> group2 routing. Initial graph built at startup. Walls filtered from FINDINGS.

nodes/collision.py -- Voronoi boundary method

Added ~35 lines after the violation cell loop. Scans Voronoi boundaries (adjacent free cells with different nearest-obstacle attribution). Real surface-to-surface gap. Overwrites min_clearance_m in obj_violation_data.

nodes/reason.py -- Context injection

Injects spatial_graph_text into LLM context before each call.

prompts.py -- SYSTEM_PROMPT update

Added SPATIAL GRAPH section instructing LLM to use move vectors from ISSUES.

nodes/add_objects.py -- Graph rebuild

Rebuilds spatial graph after both MCP and fallback placement paths. Base edges only (no analysis enrichment). try/except for graceful degradation.

MASTER_CLAUDE_V2.md -- Documentation

Updated architecture, spatial graph layer docs, component descriptions, changelog with all changes from this session.

visualize_interactive.py -- NEW - Interactive graph visualizer

Apple-minimalist HTML + vis.js 9.1.2 (~1190 lines). Fixed architectural positions, dark/light toggle, legend filtering, detail panel, drag snap-back, live auto-refresh via HTTP server (port 7477) with smart change detection and adaptive backoff fallback. Output: view_graph/spatial_graph_interactive.html.

generate_session_report.py -- This report

PDF report generator with diagrams and comprehensive change documentation.